

Redis Cache Implementation Guide - Beginner's Edition

Part 1: Why Redis Instead of Memory Cache? (The Problem)

Imagine you have a shopping cart on a website. You need to store what items the user added to their cart.

Memory Cache (IMemoryCache) - Like Your Pocket:

- Stores data in your computer's RAM
- **Problem:** If you restart your app, all data is gone!
- **Problem:** If you have multiple servers (Server A and Server B), Server A's cache is different from Server B's cache
- **Example:** User adds item on Server A, but Server B doesn't know about it

Redis Cache - Like a Shared Notebook:

- Stores data in a separate server (Redis server)
- **Benefit:** If you restart your app, data is still in Redis
- **Benefit:** All servers share the same Redis - everyone sees the same data
- **Benefit:** Super fast (in-memory storage)
- **Benefit:** Can handle millions of requests per second

Real-world example:

- **Memory Cache:** You write a note on your hand - if you wash your hands, note is gone
- **Redis:** You write a note in a shared notebook - anyone can read it, and it stays there

Why Basket Service Uses Redis:

- Shopping carts are temporary (user might not checkout)
- Need speed (carts are accessed frequently)
- Multiple instances of Basket API might run (need shared cache)

- Don't want to lose carts if server restarts

Part 2: Required NuGet Packages

You need only **ONE** main package for Redis:

Package: `Microsoft.Extensions.Caching.StackExchangeRedis` (Version 10.0.1)

What it does: Provides the interface to connect and work with Redis from .NET

Additional package (for JSON serialization): **Package:** `Newtonsoft.Json` (Version 13.0.4)

What it does: Converts C# objects to JSON strings (and back) so we can store them in Redis

Why Newtonsoft.Json?

- Redis stores data as strings
- Your ShoppingCart is a C# object
- Need to convert: C# Object → JSON String → Redis
- And when reading: Redis → JSON String → C# Object

How to install:

```
# In your Infrastructure project
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
dotnet add package Newtonsoft.Json
```

Part 3: Step-by-Step Implementation Guide

Step 1: Start Redis Server (The Foundation)

You already have Redis in your docker-compose! Let's start it:

```
# Start Redis
docker-compose up -d basketdb
```

```
# Check if it's running
```

```
docker ps
```

What this does: Starts Redis container on port 6379

Verify it's working:

```
# Connect to Redis CLI
docker exec -it basketdb redis-cli
```

```
# Test with ping
PING
# Should reply: PONG
```

```
# Exit
exit
```

Step 2: Add NuGet Package to Your Project

In your **Infrastructure** project (Basket.Infrastructure):

```
cd Services/Basket/Basket.Infrastructure
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
dotnet add package Newtonsoft.Json
```

Your .csproj file should look like this (you already have this!):

```
<ItemGroup>
  <PackageReference
    Include="Microsoft.Extensions.Caching.StackExchangeRedis"
    Version="10.0.1" />
  <PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
</ItemGroup>
```

Step 3: Configure Redis in Program.cs

In your **API project** (Basket.API/Program.cs), add Redis configuration:

```
// Add this after line 45 (after GrpcSettings)

// Redis Configuration
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
builder.Configuration.GetSection("CacheSettings")
                           .GetValue<string>("Co
nnectionString");
});
```

What this does:

- Registers Redis as a distributed cache
- Reads connection string from configuration
- Makes IDistributedCache available for dependency injection

Your current code (lines 66-70) already has this! ✓

Step 4: Add Connection String to appsettings.json

In **Basket.API/appsettings.json**:

```
{
  "CacheSettings": {
    "ConnectionString": "localhost:6379"
  }
}
```

For Docker environment (docker-compose.override.yml), it uses:

```
"CacheSettings__ConnectionString": "basketdb:6379"
```

Why two different URLs?

- localhost:6379 - When running locally (your computer)
- basketdb:6379 - When running in Docker (container name)

Step 5: Create Repository Interface

In **Basket.Core/Repositories/IBasketRepository.cs**:

```
using Basket.Core.Entities;

namespace Basket.Core.Repositories
{
    public interface IBasketRepository
    {
        Task<ShoppingCart> GetBasket(string userName);
        Task<ShoppingCart> UpsertBasket(ShoppingCart shoppingCart);
        Task DeleteBasket(string userName);
    }
}
```

What this does: Defines the contract for basket operations (you already have this! ✓)

Step 6: Implement Repository with Redis

In **Basket.Infrastructure/Repositories/BasketRepository.cs**:

```
using Basket.Core.Entities;
using Basket.Core.Repositories;
using Microsoft.Extensions.Caching.Distributed;
using Newtonsoft.Json;

namespace Basket.Infrastructure.Repositories
{
    public class BasketRepository : IBasketRepository
    {
        private readonly IDistributedCache _redisCache;

        public BasketRepository(IDistributedCache redisCache)
        {
            _redisCache = redisCache;
        }

        // GET cart from Redis
    }
}
```

```

    public async Task<ShoppingCart> GetBasket(string userName)
    {
        var basket = await _redisCache.GetStringAsync(userName);
        if (string.IsNullOrEmpty(basket))
        {
            return null;
        }
        // Convert JSON back to C# object
        return
        JsonConvert.DeserializeObject<ShoppingCart>(basket);
    }

    // DELETE cart from Redis
    public async Task DeleteBasket(string userName)
    {
        await _redisCache.RemoveAsync(userName);
    }

    // SAVE cart to Redis
    public async Task<ShoppingCart> UpsertBasket(ShoppingCart
shoppingCart)
    {
        // Convert C# object to JSON
        await _redisCache.SetStringAsync(
            shoppingCart.UserName,
            JsonConvert.SerializeObject(shoppingCart)
        );
        return await GetBasket(shoppingCart.UserName);
    }
}

```

What each method does:

- GetBasket: Reads JSON string from Redis → converts to ShoppingCart object
- UpsertBasket: Converts ShoppingCart to JSON → saves to Redis
- DeleteBasket: Removes the key from Redis

Note: Your current code has RefreshAsync on line 33, but it should be RemoveAsync for deletion!

Step 7: Register Repository in DI Container

In **Basket.API/Program.cs** (line 26):

```
builder.Services.AddScoped<IBasketRepository, BasketRepository>();
```

What this does: Makes IBasketRepository available for injection in controllers

Step 8: Use Repository in Controller

In your Basket Controller:

```
[HttpGet("{userName}")]
public async Task<ActionResult<ShoppingCart>> GetBasket(string
userName)
{
    var basket = await _basketRepository.GetBasket(userName);
    return Ok(basket);
}

[HttpPost]
public async Task<ActionResult<ShoppingCart>>
UpdateBasket(ShoppingCart basket)
{
    var updatedBasket = await _basketRepository.UpsertBasket(basket);
    return Ok(updatedBasket);
}

[HttpDelete("{userName}")]
public async Task<IActionResult> DeleteBasket(string userName)
{
    await _basketRepository.DeleteBasket(userName);
    return NoContent();
}
```

Part 4: Redis Operations - Code Examples

Basic Redis Operations

1. SET - Store data in Redis

```
// Store a simple string
await _redisCache.SetStringAsync("user:123", "John Doe");

// Store with expiration (30 minutes)
await _redisCache.SetStringAsync(
    "user:123",
    "John Doe",
    new DistributedCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)
    }
);
```

2. GET - Retrieve data from Redis

```
// Get a string
string userName = await _redisCache.GetStringAsync("user:123");
// Returns: "John Doe" or null if not found
```

3. DELETE - Remove data from Redis

```
// Remove a key
await _redisCache.RemoveAsync("user:123");
```

4. Store Complex Objects (like ShoppingCart)

```
// Convert object to JSON and store
var cart = new ShoppingCart
{
    UserName = "john",
    Items = new List<CartItem> { ... }
```

```
};
```

```
string jsonCart = JsonConvert.SerializeObject(cart);  
await _redisCache.SetStringAsync($"basket:{cart.UserName}", jsonCart);
```

5. Retrieve Complex Objects

```
// Get JSON and convert back to object  
string jsonCart = await _redisCache.GetStringAsync($"basket:john");  
var cart = JsonConvert.DeserializeObject<ShoppingCart>(jsonCart);
```

Your Current Implementation Breakdown

Looking at your BasketRepository.cs:

Line 22-28: GetBasket

```
var basket = await _redisCache.GetStringAsync(userName); // Read from  
Redis  
if (string.IsNullOrEmpty(basket))  
    return null;  
return JsonConvert.DeserializeObject<ShoppingCart>(basket); // JSON →  
Object
```

Line 36-40: UpsertBasket

```
await _redisCache.SetStringAsync(  
    shoppingCart.UserName, // Key = username  
    JsonConvert.SerializeObject(shoppingCart) // Value = JSON  
);
```

Key-Value Storage Pattern:

- **Key:** userName (e.g., "john", "rahul")
- **Value:** JSON string of ShoppingCart
- **Example:**
 - Key: "john"

- Value:
`{"UserName": "john", "Items": [{"ProductId": "123", "Quantity": 2}]}`

Redis Data Structure

Redis stores data as key-value pairs:

Redis Server:

Key: "john"
Value: {"UserName": "john", ...}
Key: "rahul"
Value: {"UserName": "rahul", ...}
Key: "farukh"
Value: {"UserName": "farukh", ...}

Advanced Redis Options

Set Expiration (Auto-delete after time)

```
await _redisCache.SetStringAsync(
    "basket:john",
    JsonConvert.SerializeObject(cart),
    new DistributedCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(24) //
Delete after 24 hours
    }
);
```

Sliding Expiration (Reset timer on access)

```
new DistributedCacheEntryOptions
{
```

```
        SlidingExpiration = TimeSpan.FromMinutes(30) // Reset timer every
time accessed
    }
```

Testing Your Redis Implementation

1. Start Redis

```
docker-compose up -d basketdb
```

2. Run your Basket API

```
cd Services/Basket/Basket.API
dotnet run
```

3. Test with Postman/curl

```
# Create basket
POST http://localhost:8001/api/v1/Basket
{
  "userName": "john",
  "items": [
    {
      "quantity": 2,
      "price": 3500,
      "productId": "602d2149e773f2a3990b47f5"
    }
  ]
}

# Get basket
GET http://localhost:8001/api/v1/Basket/john

# Delete basket
DELETE http://localhost:8001/api/v1/Basket/john
```

4. Check Redis directly

```
# Connect to Redis CLI
docker exec -it basketdb redis-cli
```

```
# List all keys
KEYS *
```

```
# Get specific key
GET "john"
```

```
# Exit
exit
```

Summary

Your Current Redis Implementation Status:  Already implemented correctly!

What you have:

-  NuGet package: `Microsoft.Extensions.Caching.StackExchangeRedis` (10.0.1)
-  NuGet package: `Newtonsoft.Json` (13.0.4)
-  Redis configuration in `Program.cs` (lines 66-70)
-  Connection string in `appsettings.json`
-  Repository implementation with Redis
-  Docker Compose with Redis container

Bug Found: In `BasketRepository.cs` line 33, you have:

```
await _redisCache.RefreshAsync(userName); // WRONG - this refreshes
expiration
```

Should be:

```
await _redisCache.RemoveAsync(userName); // CORRECT - this deletes
the key
```

Would you like me to fix this bug in your BasketRepository.cs file?

Quick Reference

Component	Purpose
Redis Server	Fast in-memory key-value store (port 6379)
StackExchangeRedis	.NET library to talk to Redis
IDistributedCache	Interface for distributed caching
Newtonsoft.Json	Converts objects to/from JSON for storage

Why Redis for Shopping Carts?

1. **Speed:** Millisecond response times
2. **Shared:** Multiple API instances see same cart data
3. **Persistent:** Survives server restarts
4. **Temporary:** Perfect for carts that might not checkout
5. **Scalable:** Handles millions of concurrent users

Your implementation follows best practices for distributed caching in microservices!